



Formation Apache Spark simplifiée

Tout passer du disque à la mémoire !



Objectifs d'apprentissage

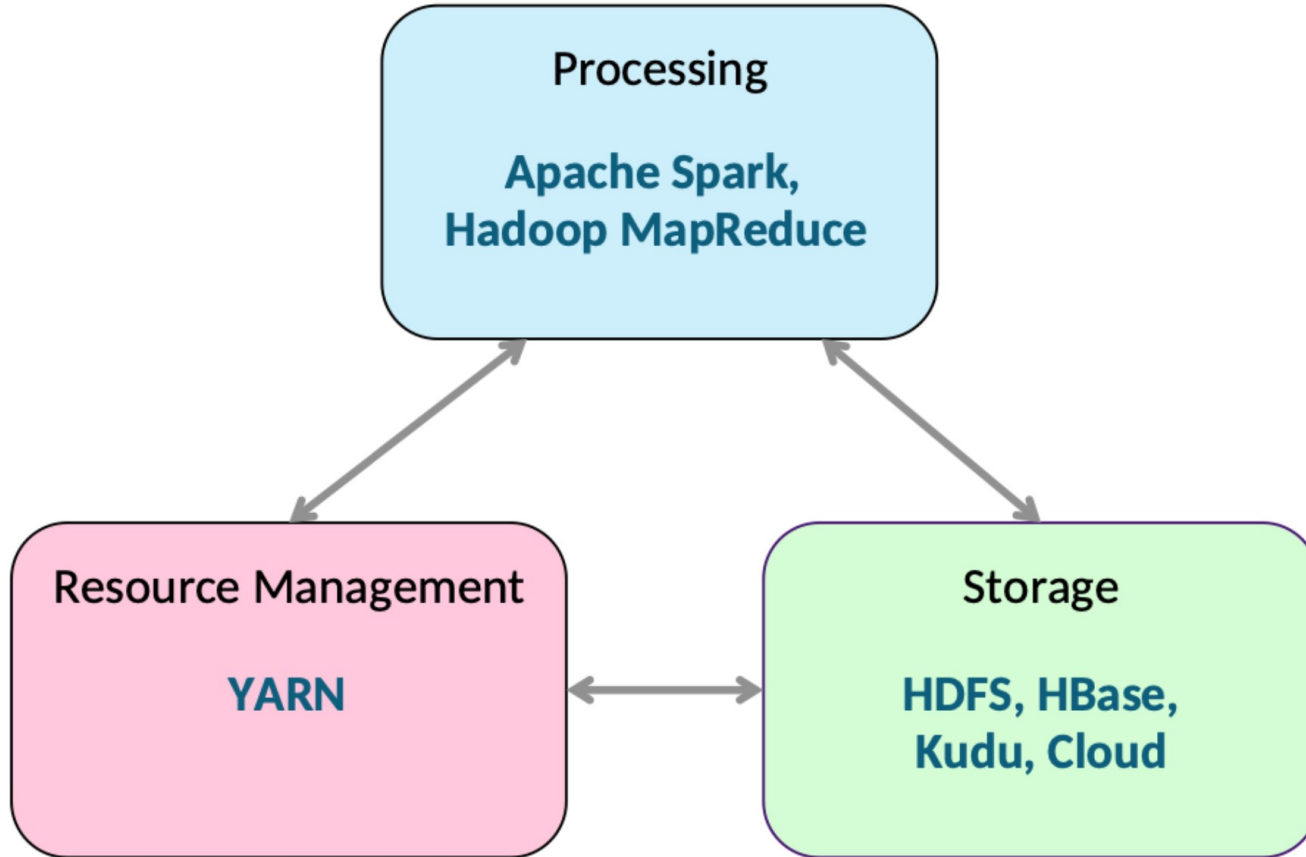
- Comprendre pourquoi Spark accélère le Big Data
- Maîtriser les concepts RDD, DataFrame & Dataset
- Écrire des traitements sans douleur (Scala/PySpark)
- Explorer des données réelles (logs & restaurants)
- Optimiser, monitorer, dépanner vos jobs

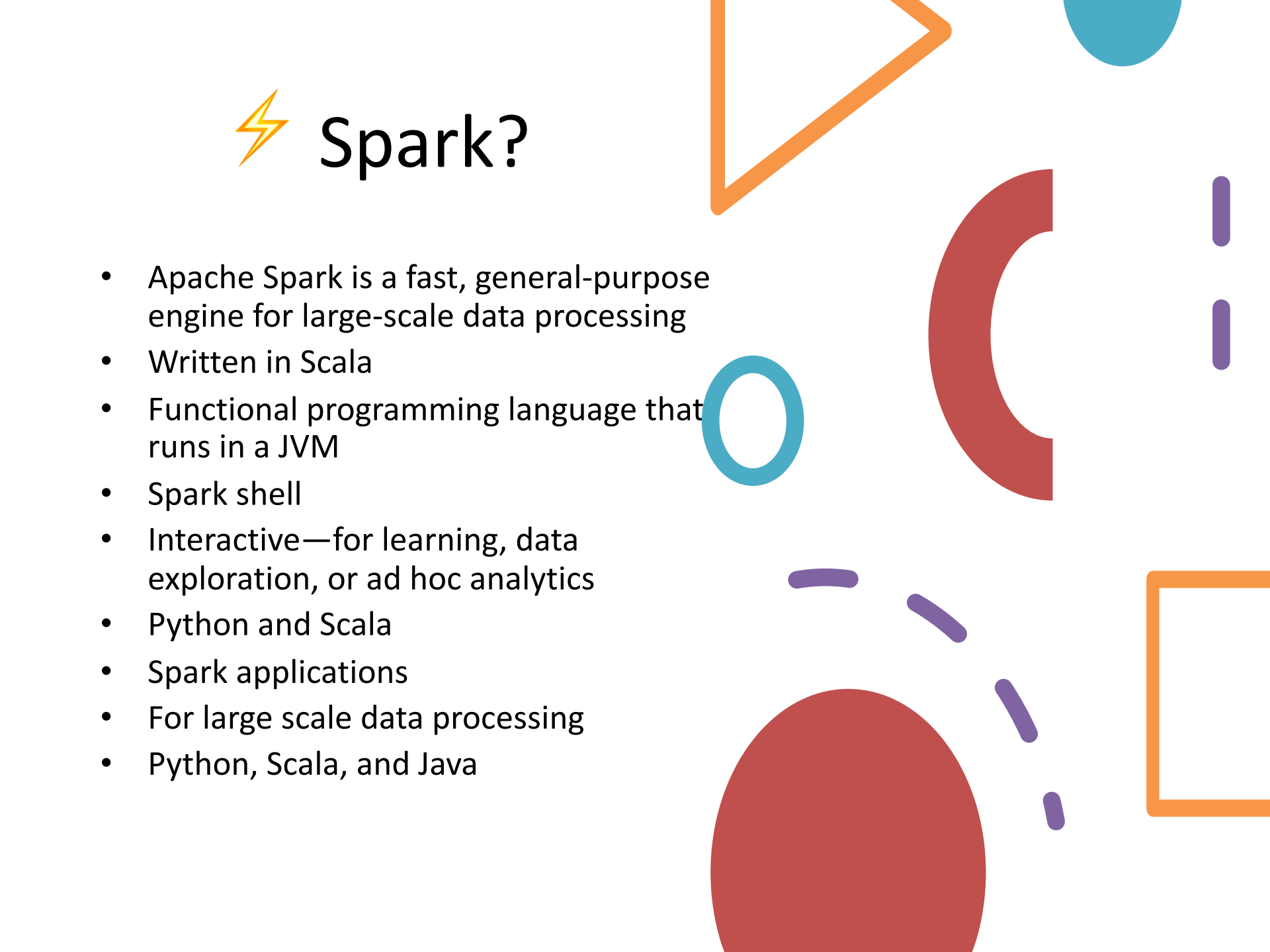


Spark vs MapReduce

- MapReduce = four traditionnel 🍞 (lent, écrit sur disque)
- Spark = micro-ondes 🚀 (tout cuit en mémoire)
- Jusqu'à 100× plus rapide sur certains algos itératifs
- API haut niveau (SQL, ML, Streaming) + multi-langages

⚡ Spark vs MapReduce



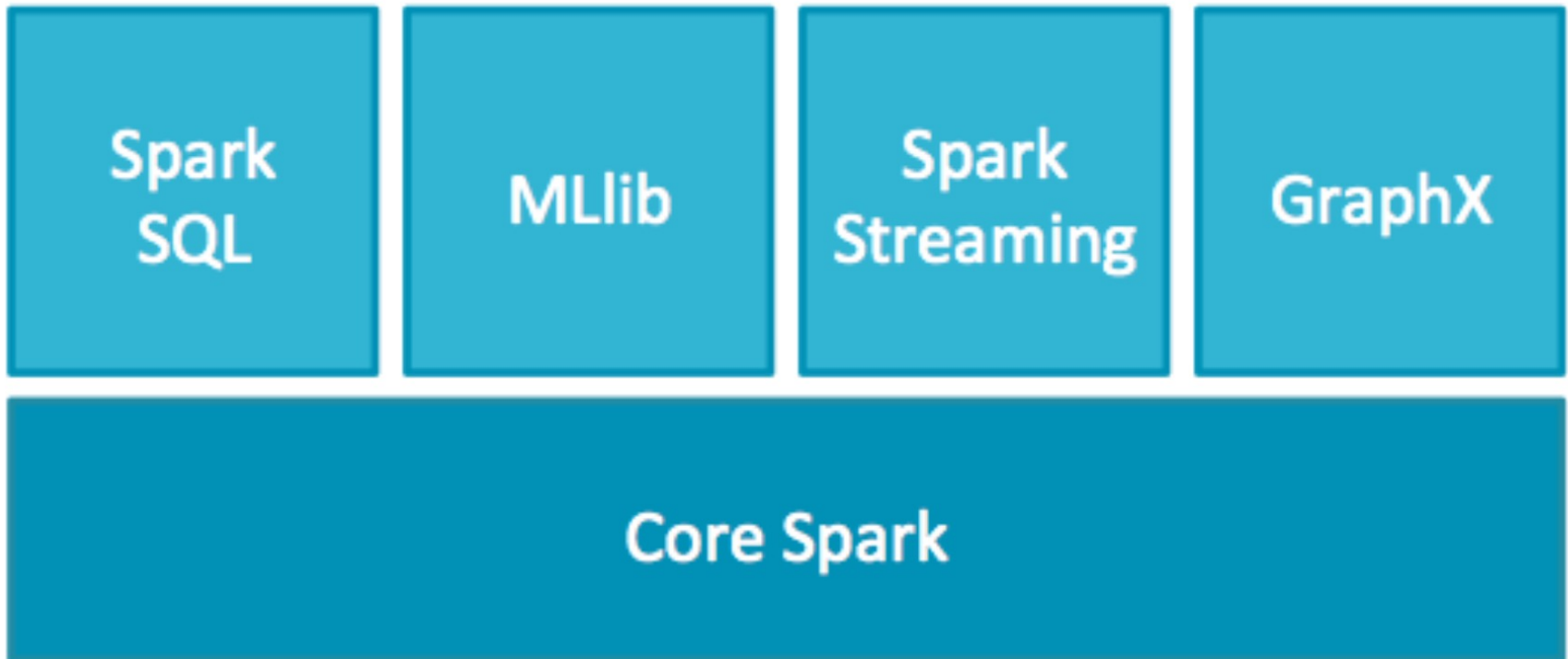


Spark?

- Apache Spark is a fast, general-purpose engine for large-scale data processing
- Written in Scala
- Functional programming language that runs in a JVM
- Spark shell
- Interactive—for learning, data exploration, or ad hoc analytics
- Python and Scala
- Spark applications
- For large scale data processing
- Python, Scala, and Java

Spark

- Spark provides a stack of libraries built on core Spark
- – Core Spark provides the fundamental Spark abstraction: Resilient Distributed
- Datasets (RDDs)
- – Spark SQL works with structured data
- – MLlib supports scalable machine learning
- – Spark Streaming applications process data in real time
- – GraphX works with graphs and graph-parallel computation



Spark SQL

- **Spark SQL is a Spark library for working with structured data**
- **What does Spark SQL provide?**
 - The DataFrame and Dataset API
 - The primary entry point for developing Spark applications
 - DataFrames and Datasets are abstractions for representing structured data
 - Catalyst Optimizer—an extensible optimization framework
 - A SQL engine and command line interface



Spark SHELL

- **The Spark shell provides an interactive Spark environment**
 - Often called a *REPL*, or Read/Evaluate/Print Loop
 - For learning, testing, data exploration, or ad hoc analytics
 - You can run the Spark shell using either Python or Scala
 - **You typically run the Spark shell on a gateway node**



Démarrage Spark SHELL

- pyspark for Python
- spark-shell for Scala
- **The Spark shell has a number of different start-up options, including**
 - master: specify the cluster to connect to
 - jars: Additional JAR files
 - py-files: Additional Python files (Python only)
 - name: the name the Spark Application UI uses for this application
 - Defaults to PySparkShell (Python) or Spark shell (Scala)
 - help: Show all the available shell options
- `$ pyspark --name "My Application"`





Spark Cluster

- **Spark applications can run on these types of clusters**
 - — Apache Hadoop YARN
 - — Kubernetes
 - — Apache Mesos
 - — Spark Standalone
- **They can also run locally instead of on a cluster**
- **CDH, HDP, and CDP Data Center use YARN**
- **CDP Private Cloud and CDP Public Cloud use Kubernetes**
- **Specify the type or URL of the cluster using the master option**
- - **Set the master option to specify cluster type**
 - yarn
 - local[*] runs locally with as many threads as cores (default)
 - local[n] runs locally with n threads
 - local runs locally with a single thread
- `$ pyspark --master yarn`



Architecture Spark

- Driver = Chef d'orchestre 
- Executors = Musiciens  répartis sur chaque machine
- Cluster manager (YARN, Standalone, Kubernetes) distribue les partitions

```
Python 3.8.10 (default, Mar 18 2025, 20:04:55)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/06/23 21:31:09 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Welcome to

  ____      _
 / ___|    / \
| |  | |  / _ \
| |  | | / ___ \
| |  | || |___|
|_|  |_| \____/
version 3.3.0

Using Python version 3.8.10 (default, Mar 18 2025 20:04:55)
Spark context Web UI available at http://localhost:4040
Spark context available as 'sc' (master = local[*], app id = local-1750707071282).
SparkSession available as 'spark'.
>>> █
```

Using Spark SHELL

- **Spark Session**
- **The main entry point for the Spark API is a Spark session**
- **The Spark shell provides a preconfigured SparkSession object called spark**

Spark Session

- Working with the Spark Session
 - The `SparkSession` class provides functions and attributes to access all of Spark functionality
 - Examples include
 - `sql`: execute a Spark SQL query
 - `catalog`: entry point for the Catalog API for managing tables
 - `read`: function to read data from a file or other data source
 - `conf`: object to manage Spark configuration settings
 - `sparkContext`: entry point for core Spark API

DataFrames and Datasets

- **DataFrames and Datasets are the primary representation of data in Spark**
 - **DataFrames represent structured data in a tabular form**
 - DataFrames model data similar to tables in an RDBMS
 - DataFrames consist of a collection of loosely typed Row objects
 - Rows are organized into columns described by a schema
- **Datasets represent data as a collection of objects of a specified type**
 - Datasets are strongly-typed—type checking is enforced at compile time rather than run time
 - An associated schema maps object properties to a table-like structure of rows and columns
 - Datasets are only defined in Scala and Java
 - **DataFrame** is an alias for **Dataset[Row]**—Datasets containing Row objects
- **DataFrames contain a collection of Row objects**
 - Rows contain an ordered collection of values
 - Row values can be basic types (such as integers, strings, and floats) or collections of those types (such as arrays and lists)
 - A schema maps column names and types to the values in a row

Example

Creating a DataFrame

- **The users.json text file contains sample data**
- — Each line contains a single JSON record that can include a name, age, and
- postal code field

```
{"name":"Alice", "pcode":"94304"}  
{"name":"Brayden", "age":30, "pcode":"94304"}  
{"name":"Carla", "age":19, "pcode":"10036"}  
{"name":"Diana", "age":46}  
{"name":"Étienne", "pcode":"94104"}
```

**Create a DataFrame
using spark.read**

```
usersDF = spark.read.json("file:///tmp/users.json")
```

**Returns the
Spark session's
DataFrameReader**

**Call json function
to create a new
DataFrame**

- **DataFrames always have an associated schema**

- **DataFrameReader can infer the schema from the data**

- **Use printSchema to show the DataFrame's schema**

```
usersDF = spark.read.json("file:///tmp/users.json")
>>> usersDF.printSchema()
root
|-- age: long (nullable = true)
|-- name: string (nullable = true)
|-- pcode: string (nullable = true)
```

**The show method
displays the first few
rows in a tabular
format**

```
usersDF = spark.read.json("file:///tmp/users.json")
>>> usersDF.printSchema()
root
 |-- age: long (nullable = true)
 |-- name: string (nullable = true)
 |-- pcode: string (nullable = true)
```

```
usersDF.show()
```

```
>>> usersDF = spark.read.json("file:///tmp/users.json")
>>>
>>> usersDF.printSchema()
root
 |-- age: long (nullable = true)
 |-- name: string (nullable = true)
 |-- pcode: string (nullable = true)

>>> usersDF.show()
+-----+-----+-----+
| age|   name|pcode|
+-----+-----+-----+
| null|  Alice|94304|
|  30|Brayden|94304|
|  19|  Carla|10036|
|  46|  Diana|  null|
| null|Étienne|94104|
+-----+-----+-----+
```

DataFrames

Opérations

- There are two main types of DataFrame operations
 - *Transformations* create a new DataFrame based on existing one(s)
 - Transformations are executed in parallel by the application's executors
 - *Actions* output data values from the DataFrame
 - Output is typically returned from the executors to the main Spark
- program (called the *driver*) or saved to a file

DataFrames

Opérations:

Actions

- **Some common DataFrame actions include**
 - **count**: returns the number of rows
 - **first**: returns the first row (synonym for `head()`)
 - **take(n)**: returns the first n rows as an array (synonym for `head(n)`)
 - **show(n)**: display the first n rows in tabular form (default is 20 rows)
 - **collect**: returns all the rows in the DataFrame as an array
 - **write**: save the data to a file or other data source

DataFrames

Opérations:

Actions

```
users = usersDF.take(3)  
print(users)
```

```
[Row(age=None, name=u'Alice', pcode=u'94304'),  
Row(age=30, name=u'Brayden', pcode=u'94304'),  
Row(age=19, name=u'Carla', pcode=u'10036')]
```

DataFrames Opérations: Transformations



**Transformations
create a new
DataFrame
based on an
existing one**

The new DataFrame
may have the same
schema or a different
one



**Transformations
do not return
any values or
data to the
driver**

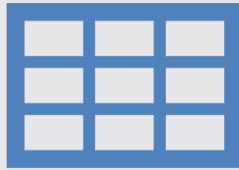
Data remains
distributed across the
application's
executors



**DataFrames are
immutable**

Data in a DataFrame
is never modified
Use transformations
to create a new
DataFrame with the
data you need

DataFrames Opérations: Transformations



Common transformations include

`select`: only the specified columns are included

`where`: only rows where the specified expression is true are included



(synonym for filter)

`orderBy`: rows are sorted by the specified column(s) (synonym for `sort`)

`join`: joins two DataFrames on the specified column(s)

`limit(n)`: creates a new DataFrame with only the first `n` rows

DataFrames

Opérations:

Transformations

```
nameAgeDF = usersDF.select("name","age")
```

```
> nameAgeDF.show()
```

```
+-----+-----+
```

```
| name| age|
```

```
+-----+-----+
```

```
| Alice| null|
```

```
| Brayden| 30|
```

```
| Carla| 19|
```

```
| Diana| 46|
```

```
| Etienne| null|
```

```
+-----+-----+
```

```
> over20DF = usersDF.where("age > 20")
```

```
> over20DF.show()
```

```
+---+-----+-----+
```

```
| age| name| code|
```

```
+---+-----+-----+
```

```
| 30| Brayden| 94304|
```

```
| 46| Diana| null|
```

```
+---+-----+-----+
```


DataFrames Opérations: Transformations

A sequence of transformations followed
by an action is a *query*

```
nameAgeDF = usersDF.select("name","age")
> nameAgeOver20DF =
nameAgeDF.where("age > 20")
> nameAgeOver20DF.show()
+---+-----+
|age| name|
+---+-----+
| 30|Brayden|
| 46|Diana|
+---+-----+
```

DataFrames Opérations: Transformations

A sequence of transformations followed
by an action is a *query*

Transformations in a query can be chained together
These two examples perform the same query in the
same way

- Differences are only syntactic

```
nameAgeDF = usersDF.select("name","age")
nameAgeOver20DF = nameAgeDF.where("age > 20")
nameAgeOver20DF.show()
usersDF.select("name","age").where("age > 20").show()
```

Analyzing Data with DataFrame Queries

Columns, Column Names, and Column Expressions

- Most DataFrame transformations require you to specify a column or columns
 - `select(column1, column2, ...)`
 - `orderBy(column1, column2, ...)`
- For many simple queries, you can just specify the column name as a string
 - `peopleDF.select("firstName","lastName")`
- Some types of transformations use column references or column expressions instead of column name strings



```
def select(col: String, cols: String*): DataFrame  
    Selects a set of columns.
```

```
def select(cols: Column*): DataFrame  
    Selects a set of column based expressions.
```

Analyzing Data with DataFrame Queries

```
peopleDF =  
spark.read.option("header", "true").csv("file:///tmp/people.  
csv")  
peopleDF['age']  
Column<age>  
peopleDF.age  
Column<age>  
peopleDF.select(peopleDF.age).show()
```



Analyzing Data with DataFrame Queries

Using column references instead of simple strings allows you to create *column expressions*

- **Column operations include**

- Arithmetic operators such as +, -, %, /, and *
- Comparative and logical operators such as >, <, && and ||
- The equality comparator is == in Python
- String functions such as contains, like, and substring
- Data testing functions such as isNull, isNotNull, and NaN (not a number)
- Sorting functions such as asc and desc
- Work only when used in sort/orderBy



Analyzing Data with DataFrame Queries

```
peopleDF.select("lastName", peopleDF.age * 10).show()
```

```
+-----+
|lastName|(age * 10)|
+-----+
| Davis| 340.0|
| Roach| 430.0|
| Bryant| 480.0|
| Williams| 420.0|
| King| 640.0|
| McIntyre| 580.0|
| Cruz| 570.0|
| Velez| 420.0|
| Nunez| 430.0|
| Campbell| 230.0|
| Watts| 180.0|
| Johnson| 210.0|
| Savage| 450.0|
| Murphy| 410.0|
| Hudson| 310.0|
| Crosby| 630.0|
| Adams| 310.0|
| Porter| 620.0|
| Watts| 320.0|
| Martinez| 530.0|
+-----+
```

```
peopleDF.where(peopleDF.firstName.startswith("A")).show()
```

```
+-----+-----+-----+
|age|firstName|lastName|pcode|
+-----+-----+-----+
| 56| Amber| Mirandal| 75605|
```



Analyzing Data with DataFrame Queries

Use the column alias function to rename a column in a result set

— name is a synonym for alias

```
peopleDF.select("lastName",  
(peopleDF.age * 10).alias("age_10")).show()
```



Analyzing Data with DataFrame Queries

Grouping and Aggregation Queries

Aggregation queries perform a calculation on a set of values and return a single value

- To execute an aggregation on a set of grouped values, use `groupBy` combined with an aggregation function
- Example: How many people are in each postal code?

```
peopleDF.groupBy("pcode").count().show()
```

```
+----+----+
|pcode|count|
+----+----+
|75219|  5|
|75967|  2|
|75261|  8|
|75628|  3|
|75963|  8|
|75355|  7|
|75680|  3|
|75241|  6|
|75986|  5|
|75605|  3|
+----+----+
```



Analyzing Data with DataFrame Queries

The groupBy Transformation

- **groupBy takes one or more column names or references**
 - In Python, returns a GroupedData object
- **Returned objects provide aggregation functions, including**
 - count
 - max and min
 - mean (and its alias avg)
 - sum
 - pivot
 - agg (aggregates using additional aggregation functions)



Analyzing Data with DataFrame Queries

The functions **object** provides several additional aggregation functions

- **Aggregate functions include**

- `first/last` returns the first or last items in a group
- `countDistinct` returns the number of unique items in a group
- `approx_count_distinct` returns an approximate counts of unique items
- Much faster than a full count
- `stddev` calculates the standard deviation for a group of values
- `var_sample/var_pop` calculates the variance for a group of values
- `covar_samp/covar_pop` calculates the sample and population covariance of a group of values
- `corr` returns the correlation of a group of values



Analyzing Data with DataFrame Queries

```
>>> from pyspark.sql.functions import stddev
```

```
peopleDF.groupBy("pcode").agg(stddev("age")).show()
```

```
+-----+-----+  
|pcode| stddev_samp(age)|  
+-----+-----+  
|75219| 13.47219358530748|  
|75967| 16.263455967290593|  
|75261| 11.80118032837636|  
|75628| 3.605551275463989|  
|75963| 13.81510560427038|  
|75355| 13.624208146495986|  
|75680| 18.24828759089466|  
|75241| 14.497126151988423|  
|75986| 18.660117898877274|  
|75605| 10.598742063723098|  
+-----+-----+
```



Analyzing Data with DataFrame Queries

Joining DataFrames

- Use the join transformation to join two DataFrames
- DataFrames support several types of joins
 - inner (default)
 - outer
 - left_outer
 - right_outer
 - leftsemi
- The crossJoin transformation joins every element of one DataFrame with every element of the other



Analyzing Data with DataFrame Queries

- This example shows an inner join when join column is in both DataFrames

```
+-----+-----+-----+-----+
|pcode|age|firstName|lastName|      city|state|
+-----+-----+-----+-----+
|75241| 34|  Erin|  Davis|  Paynechester| IA|
|75963| 43| Kenneth| Roach|  Gregoryhaven| VT|
|75963| 48|  Julie| Bryant|  Gregoryhaven| VT|
|75219| 42| Kimberly|Williams| Singletonville| HI|
|75605| 64|  Heidi|  King|  Karenside| MI|
|75963| 58| Jordan|Mcintyre|  Gregoryhaven| VT|
|75628| 57| Joshua|  Cruz|  Fritztown| NH|
|75967| 42|  Debra| Velez|  South Cynthia| PA|
|75605| 43| Jonathan| Nunez|  Karenside| MI|
|75261| 23| Michele|Campbell|  Myerschester| AK|
|75986| 18| Thomas| Watts|  Tiffanymouth| OR|
|75219| 21| Kristin| Johnson| Singletonville| HI|
|75241| 45| Peter| Savage|  Paynechester| IA|
```

Specify type of join as inner (default), outer, left_outer, right_outer, or leftsemi
`peopleDF.join(pcodesDF, "pcode", "left_outer").show()`

`peopleDF.join(zcodesDF, peopleDF.pcode == zcodesDF.zip).show()`





Resilient Distributed Datasets (RDDs)

- RDDs are part of core Spark

- *Resilient Distributed Dataset* (RDD)

- *Resilient*: If data in memory is lost, it can be recreated

- *Distributed*: Processed across the cluster

- *Dataset*: Initial data can come from a source such as a file, or it can be created programmatically

- Despite the name, RDDs are *not* Spark SQL Dataset objects

- RDDs predate Spark SQL and the DataFrame/Dataset API

Comparing RDDs to DataFrames and Datasets

Comparing RDDs to DataFrames and Datasets

- **RDDs are unstructured**
 - No schema defining columns and rows
 - Not table-like; cannot be queried using SQL-like transformations such as where and select
 - RDD transformations use lambda functions
- **RDDs can contain any type of object**
 - DataFrames are limited to Row objects
 - Datasets are limited to Row objects, case class objects (products), and primitive types

Comparing RDDs to DataFrames and Datasets

RDDs are used in all Spark languages (Python, Scala, Java)

- Strongly-typed in Scala and Java, like Datasets
 - **RDDs are not optimized by the Catalyst optimizer**
- Manually coded RDDs are typically less efficient than DataFrames
 - **You can use RDDs to create DataFrames and Datasets**
- RDDs are often used to convert unstructured or semi-structured data into structured form
- You can also work directly with the RDDs that underlie DataFrames and Datasets

RDD Data Types

- **RDDs can hold any serializable type of element**

- Primitive types such as integers, characters, and booleans
- Collections such as strings, lists, arrays, tuples, and dictionaries (including nested collection types)
- Scala/Java Objects (if serializable)
- Mixed types

- **Some RDDs are specialized and have additional functionality**

- Pair RDDs
- RDDs consisting of key-value pairs
- Double RDDs
- RDDs consisting of numeric data

- **There are several types of data sources for RDDs**

- Files, including text files and other formats
- Data in memory
- Other RDDs
- Datasets or DataFrames

Creating RDDs from Files

- **Use SparkContext object, not SparkSession**
 - SparkContext is part of the core Spark library
 - SparkSession is part of the Spark SQL library
 - One Spark context per application
 - Use SparkSession.sparkContext to access the Spark context
 - Called SC in the Spark shell
- **Create file-based RDDs using the Spark context**
 - Use `textFile` or `wholeTextFiles` to read text files
 - Use `hadoopFile` or `newAPIHadoopFile` to read other formats

Creating RDDs from Text Files

- **SparkContext.textFile reads newline-terminated text files**

- Accepts a single file, a directory of files, a wildcard list of files, or a comma-separated list of files

- Examples

- `textFile("myfile.txt")`

- `textFile("mydata/")`

- `textFile("mydata/*.log")`

- `textFile("myfile1.txt,myfile2.txt")`

```
myRDD = spark.sparkContext.textFile("file:///tmp/")
```

Creating RDDs from Text Files

textFile maps each line in a file to a separate RDD element

— Only supports newline-terminated text

```
myRDD = spark.sparkContext.textFile("file:///tmp/purplecow.txt")  
myRDD.take(10) # ou myRDD.collect() ou myRDD.foreach(print)
```

Multi-line Text Elements

- **textFile maps each line in a file to a separate RDD element**
- What about files with a multi-line input format, such as XML or JSON?
- **Use wholeTextFiles**
- Maps entire contents of each file in a directory to a single RDD element
- Works only for small files (element must fit in memory)

user1.json

```
{  
  "firstName": "Fred",  
  "lastName": "Flintstone",  
  "userid": "123"  
}
```

user2.json

```
{  
  "firstName": "Barney",  
  "lastName": "Rubble",  
  "userid": "234"  
}
```

Saving RDDs

- You can save RDDs to the same data source types supported for reading RDDs

- Use `RDD.saveAsTextFile` to save as plain text files in the specified directory

- Use `RDD.saveAsHadoopFile` or `saveAsNewAPIHadoopFile` with a specified Hadoop `OutputFormat` to save using other formats

- The specified save directory cannot already exist

```
myRDD.saveAsTextFile("mydata/")
```

RDD Operations

- **Two general types of RDD operations**

- *Actions* return a value to the Spark driver or save data to a data source

- *Transformations* define a new RDD based on the current one(s)

- **RDDs operations are performed lazily**

- Actions trigger execution of the base RDD transformations

RDD Actions & Operations

RDD Action Operations

■ Some common actions

- count returns the number of elements
- first returns the first element
- take(n) returns list (Python) of the first n elements
- collect returns list (Python) of all elements
- saveAsTextFile(dir) saves to text files

RDD Transformation Operations

- Transformations create a new RDD from an existing one
- RDDs are immutable
 - Data in an RDD is never changed
 - Transform data to create a new RDD
- A transformation operation executes a *transformation function*
 - The function transforms elements of an RDD into new elements
 - Some transformations implement their own transformation logic
 - For many, you must provide the function to perform the transformation

RDD Transformation Operations

■ Transformation operations include

- `distinct` creates a new RDD with duplicate elements in the base RDD removed
- `union(rdd)` creates a new RDD by appending the data in one RDD to another
- `map(function)` creates a new RDD by performing a function on each record in the base RDD
- `filter(function)` creates a new RDD by including or excluding each record in the base RDD according to a Boolean function

Example: distinct and union Transformations

cities1.csv

Boston,MA

Palo Alto,CA

Santa Fe,NM

Palo Alto,CA

cities2.csv

Calgary,AB

Chicago,IL

Palo Alto,CA

```
# Lire le premier fichier et enlever les doublons
distinctRDD = sc.textFile("file:///tmp/cities1.csv").distinct()
distinctRDD.foreach(print)
```

```
# Lire un deuxième fichier et faire l'union avec le premier
unionRDD =
sc.textFile("file:///tmp/cities2.csv").union(distinctRDD)
unionRDD.foreach(print)
```

Transforming Data with RDDs

- **Key concepts of *functional programming***

- Functions are the fundamental unit of programming
- Functions have input and output only
- No state or side effects
- Functions can be passed as arguments to other functions
- Called *procedural parameters*

- **Spark's architecture is based on functional programming**

- Passed functions can be executed by multiple executors in parallel

Transforming Data with RDDs

- **RDD transformations execute a *transformation procedure***
 - Transforms elements of an RDD into new elements
 - Runs on executors
- **A few transformation operations implement their own transformation logic**
 - Examples: distinct and union
- **Most transformation operations require you to pass a function**
 - Function implements your own transformation procedure
 - Examples: map and filter
- **This is a key difference between RDDs and DataFrame/Datasets**

Passing Functions

- ■ Passed functions can be named or anonymous
- ■ Anonymous functions are defined inline without an identifier
 - — Best for short, one-off functions
 - — Supported in many programming languages
 - — Python: `lambda x: ...`
 - — Scala: `x => ...`
 - — Java 8: `x -> ...`

`myUpperRDD = myRDD.map(toUpper)`

I've never seen a purple cow => I'VE NEVER SEEN A PURPLE COW.

Example: Passing Anonymous Functions

Python: Use the `lambda` keyword to specify the name of the input parameter(s) and the function that returns the output

```
myUpperRDD = myRDD.map(lambda line: line.upper())
```

Example:

```
myFilteredRDD = myRDD.map(lambda line: line.upper()).filter(lambda line: line.startswith('I'))
```

RDD Execution

An RDD query consists of a sequence of one or more transformations completed by an action

- **RDD queries are executed *lazily***

- When the action is called

- **RDD queries are executed differently than DataFrame and Dataset queries**

- DataFrames and Datasets scan their sources to determine the schema *eagerly* (when created)

- RDDs do not have schemas and do not scan their sources before loading

RDD Lineage

- **Transformations create a new RDD based on one or more existing RDDs**
 - Result RDDs are considered *children* of the base (*parent*) RDD
 - Child RDDs depend on their parent RDD
- **An RDD's *lineage* is the sequence of ancestor RDDs that it depends on**
 - When an RDD executes, it executes its lineage starting from the source
- **Spark maintains each RDD's lineage**
 - Use `toDebugString` to view the lineage

PipeLine

When possible, Spark will perform sequences of transformations by element so no data is stored

```
myFilteredRDD = sc.textFile("file:///tmp/purplecow.txt") \  
.map(lambda line: line.upper()) \  
.filter(lambda line: line.startswith("I"))
```

```
myFilteredRDD.take(2)
```

Converting RDDs to DataFrames

- **You can create a DataFrame from an RDD**

- Useful with unstructured or semi-structured data such as text
- Define a schema
- Transform the base RDD to an RDD of Row objects (Scala) or lists (Python)
- Use `SparkSession.createDataFrame`

- **You can also return the underlying RDD of a DataFrame**

- Use the `DataFrame.rdd` attribute to return an RDD of Row objects

Example

```
from pyspark.sql.types import *

mySchema = StructType([
    StructField("pcode", StringType()),
    StructField("lastName", StringType()),
    StructField("firstName", StringType()),
    StructField("age", IntegerType())
])

myRDD = sc.textFile("file:///tmp/people.csv") \
    .map(lambda line: line.split(",")) \
    .map(lambda values: [values[0], values[1], values[2], int(values[3])])

myDF = spark.createDataFrame(myRDD, mySchema)
myDF.show(2)
```

Example: Return a DataFrame's Underlying RDD

```
myRDD2 = myDF.rdd  
for row in myRDD2.take(2): print(row)  
Row(pcode=u'02134', lastName=u'Hopper', firstName=u'Grace',  
age=52)  
Row(pcode=u'94020', lastName=u'Turing', firstName=u'Alan',  
age=32)
```

Aggregating Data with Pair RDDs

Pair RDDs

- **Pair RDDs are a special form of RDD**

- Each element must be a key/value pair
(a two-element *tuple*)

- Keys and values can be any type

- **Why?**

- Use with map-reduce algorithms

- Many additional functions are
available for common data processing
needs

- Such as sorting, joining, grouping,
and counting

(key1,value1)

(key2,value2)

(key3,value3)

Creating Pair RDDs

- **The first step in most workflows is to get the data into key/value form**

- What should the RDD should be keyed on?

- What is the value?

- **Commonly used functions to create pair RDDs**

- map

- flatMap/flatMapValues

- keyBy

Example: A Simple Pair RDD (Python)

- `usersRDD = sc.textFile("file:///tmp/userlist.tsv"). \`
- `map(lambda line: line.split('\t')). \`
- `map(lambda fields: (fields[0],fields[1]))`

```
user001\tFred Flintstone  
user090\tBugs Bunny  
user111\tHarry Potter  
...
```

→

<code>("user001", "Fred Flintstone")</code>
<code>("user090", "Bugs Bunny")</code>
<code>("user111", "Harry Potter")</code>
...

Question 1: Pairs with Complex Values

■ How would you do this?

- Input: a tab-delimited list of postal codes with latitude and longitude
- Output: **postal code** (key) and **lat/long** pair (value)

```
00210\t43.00589\t-71.01320
01014\t42.17073\t-72.60484
01062\t42.32423\t-72.67915
01263\t42.3929\t-73.22848
...
```

→

("00210", (43.00589, -71.01320))
("01014", (42.17073, -72.60484))
("01062", (42.32423, -72.67915))
("01263", (42.3929, -73.22848))
...

Answer 1: Pairs with Complex Values

```
sc.textFile("file:///tmp/latlon.tsv"). \
map(lambda line: line.split('\t')). \
map(lambda fields:
    (fields[0],(float(fields[1]),float(fields[2]))))
```

```
val ordersRDD =
sc.textFile("orderskus.txt").
map(line => line.split(' '))
```

```
00210\t43.00589\t-71.01320
01014\t42.17073\t-72.60484
01062\t42.32423\t-72.67915
01263\t42.3929\t-73.22848
...
```

→

("00210", (43.00589, -71.01320))
("01014", (42.17073, -72.60484))
("01062", (42.32423, -72.67915))
("01263", (42.3929, -73.22848))
...

Question 2: Mapping Single Elements to Multiple Pairs

How would you do this?

- Input: order numbers with a list of SKUs in the order
- Output: **order** (key) and **sku** (value)

```
00001 sku010:sku933:sku022
00002 sku912:sku331
00003 sku888:sku022:sku010:sku594
00004 sku411
```

→

("00001", "sku010")
("00001", "sku933")
("00001", "sku022")
("00002", "sku912")
("00002", "sku331")
("00003", "sku888")
...

Answer 2: Mapping Single Elements to Multiple Pairs

```
ordersRDD = sc.textFile("file:///tmp/orderskus.txt") \  
  .map(lambda line: line.split(' ')) \  
  .map(lambda fields: (fields[0], fields[1])) \  
  .flatMapValues(lambda skus: skus.split(':'))
```

```
for row in ordersRDD.collect():  
    print(row)
```

```
00001 sku010:sku933:sku022  
00002 sku912:sku331  
00003 sku888:sku022:sku010:sku594  
00004 sku411
```

→

("00001", "sku010")

("00001", "sku933")

("00001", "sku022")

("00002", "sku912")

("00002", "sku331")

("00003", "sku888")

...

Map Reduce

Map-reduce in Spark works on pair RDDs

■ Map phase

- Operates on one record at a time
- “Maps” each record to zero or more new records
- Examples: map, flatMap, filter, keyBy

■ Reduce phase

- Works on map output
- Consolidates multiple records
- Examples: reduceByKey, sortByKey, mean

Map-Reduce Example: Word Count

Input Data

the cat sat on the mat
the aardvark sat on the sofa

?
→

Result

(on, 2)

(sofa, 1)

(mat, 1)

(aardvark, 1)

(the, 4)

(cat, 1)

(sat, 2)

Example: Word Count (1)

```
countsRDD = sc.textFile("file:///tmp/ catsat.txt").flatMap(lambda line: line.split(' '))
```

the
cat
sat
on
the
mat
the
aardvark
...

Example: Word Count (2)

```
countsRDD = sc.textFile("file:///tmp/ catsat.txt").flatMap(lambda line: line.split(' ')).  
map(lambda word: (word,1))
```

the
cat
sat
on
the
mat
the
aardvark
...

(the, 1)
(cat, 1)
(sat, 1)
(on, 1)
(the, 1)
(mat, 1)
(the, 1)
(aardvark, 1)
...

Example: Word Count (3)

```
countsRDD = sc.textFile("file:///tmp/ catsat.txt").flatMap(lambda line: line.split(' ')).  
map(lambda word: (word,1)).reduceByKey(lambda v1,v2: v1+v2)
```

the
cat
sat
on
the
mat
the
aardvark
...

(the, 1)
(cat, 1)
(sat, 1)
(on, 1)
(the, 1)
(mat, 1)
(the, 1)
(aardvark, 1)
...

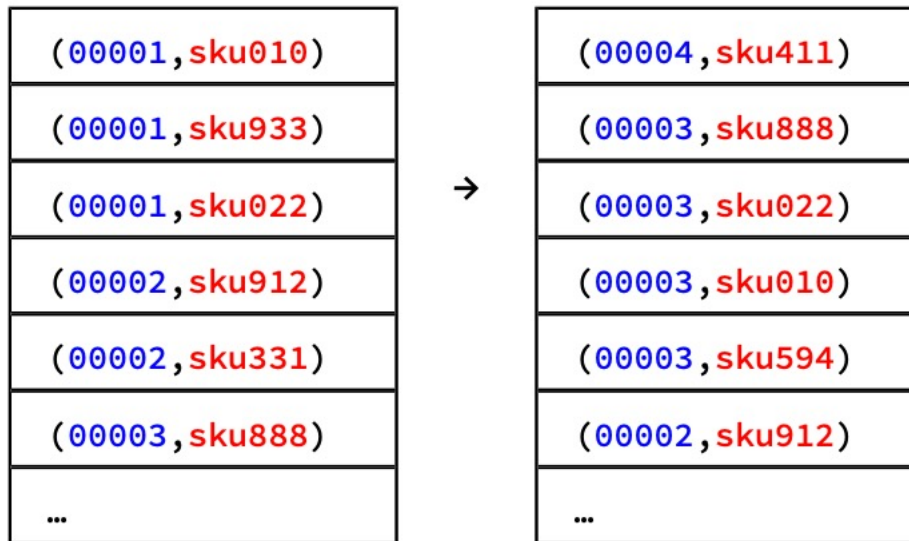
(on, 2)
(sofa, 1)
(mat, 1)
(aardvark, 1)
(the, 4)
(cat, 1)
(sat, 2)

Pair RDD Operations

- In addition to map and reduceByKey operations, Spark has several operations specific to pair RDDs
- Examples
 - countByKey returns a map with the count of occurrences of each key
 - groupByKey groups all the values for each key in an RDD
 - sortByKey sorts in ascending or descending order
 - join returns an RDD containing all pairs with matching keys from two RDDs
 - leftOuterJoin, rightOuterJoin , fullOuterJoin join two RDDs, including keys defined in the left, right, or both RDDs respectively
 - mapValues, flatMapValues execute a function on just the values, keeping the key the same
 - lookup(key) returns the value(s) for a key as a list

Example: sortByKey Transformation

- ordersRDD.sortByKey(ascending=false)



Example: groupByKey Transformation

ordersRDD.groupByKey()

(00001, sku010)	→	(00002, [sku912, sku331])
(00001, sku933)		(00001, [sku022, sku010, sku933])
(00001, sku022)		(00003, [sku888, sku022, sku010, sku594])
(00002, sku912)		(00004, [sku411])
(00002, sku331)		
(00003, sku888)		
...		

Example: joining ByKey Transformation

movieGrossRDD

(Casablanca, \$3.7M)
(Star Wars, \$775M)
(Annie Hall, \$38M)
(Argo, \$232M)
...



movieYearRDD

(Casablanca, 1942)
(Star Wars, 1977)
(Annie Hall, 1977)
(Argo, 2012)
...

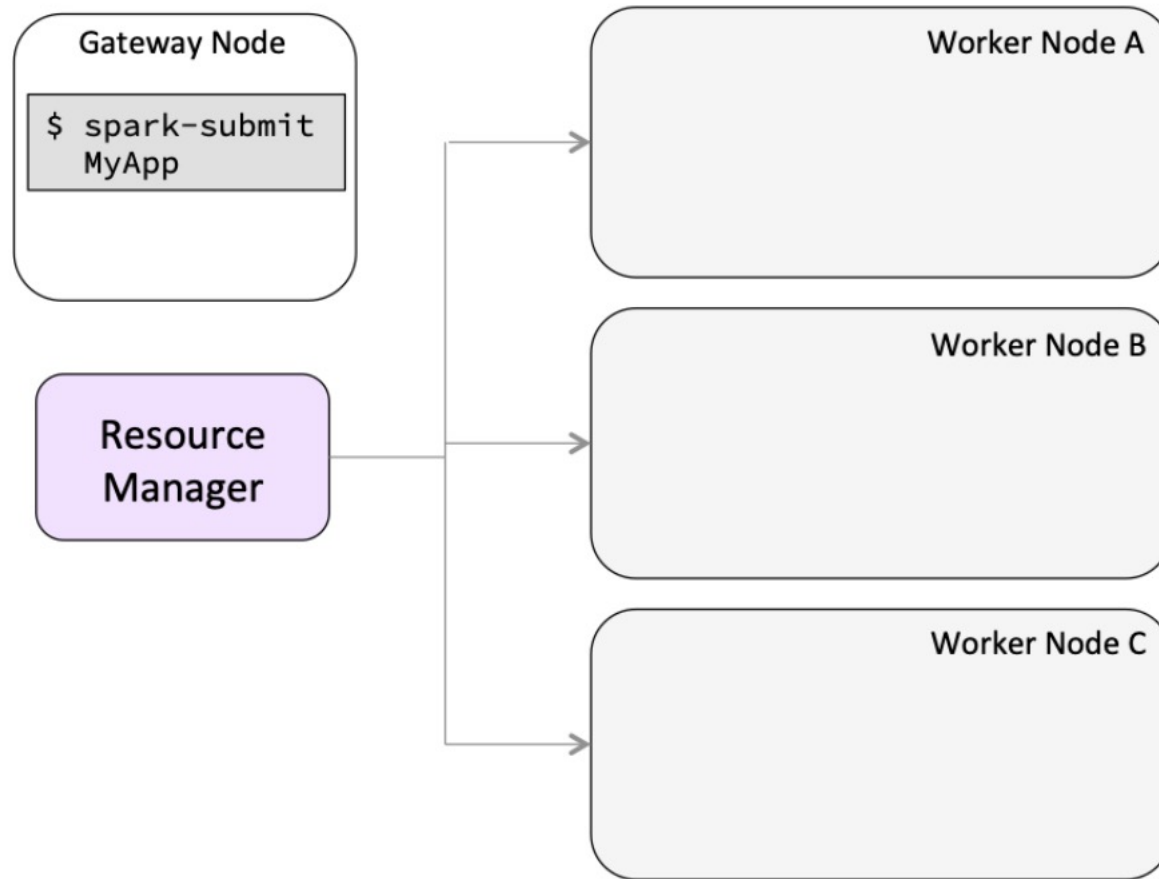
joinedRDD = movieGrossRDD.
join(movieYearRDD)

joinedRDD

(Casablanca, (\$3.7M, 1942))
(Star Wars, (\$775M, 1977))
(Annie Hall, (\$38M, 1977))
(Argo, (\$232M, 2012))
...

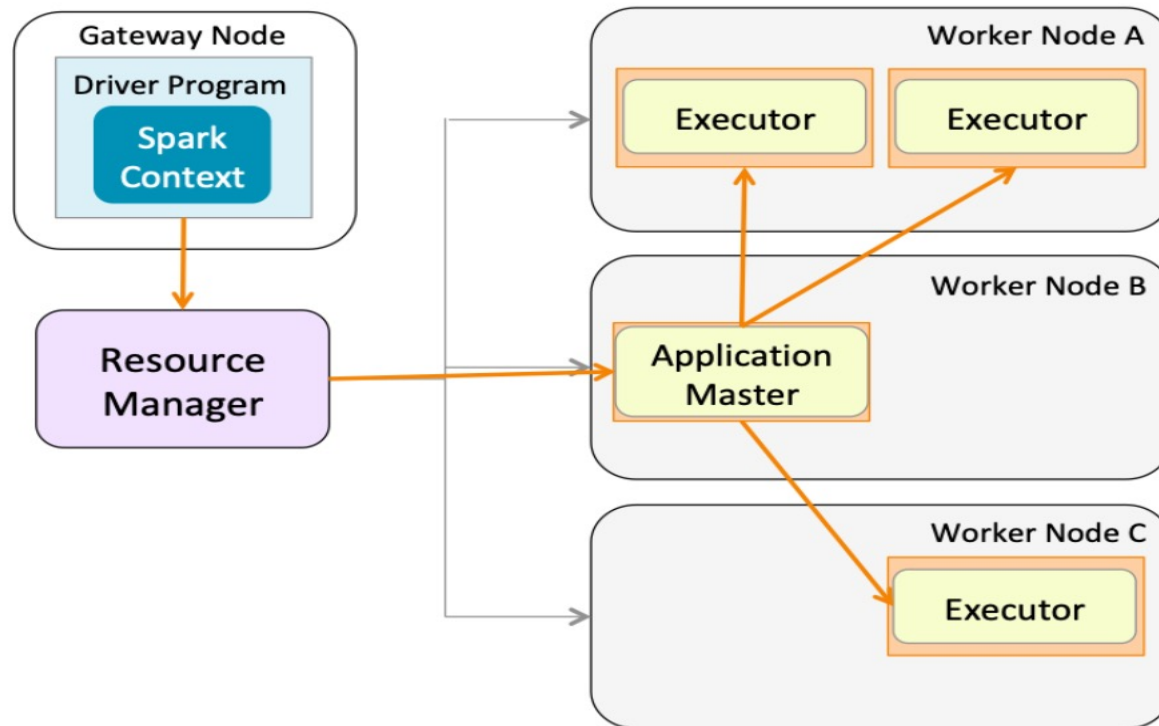
Spark Distributed Processing

Review of Spark on YARN (1)



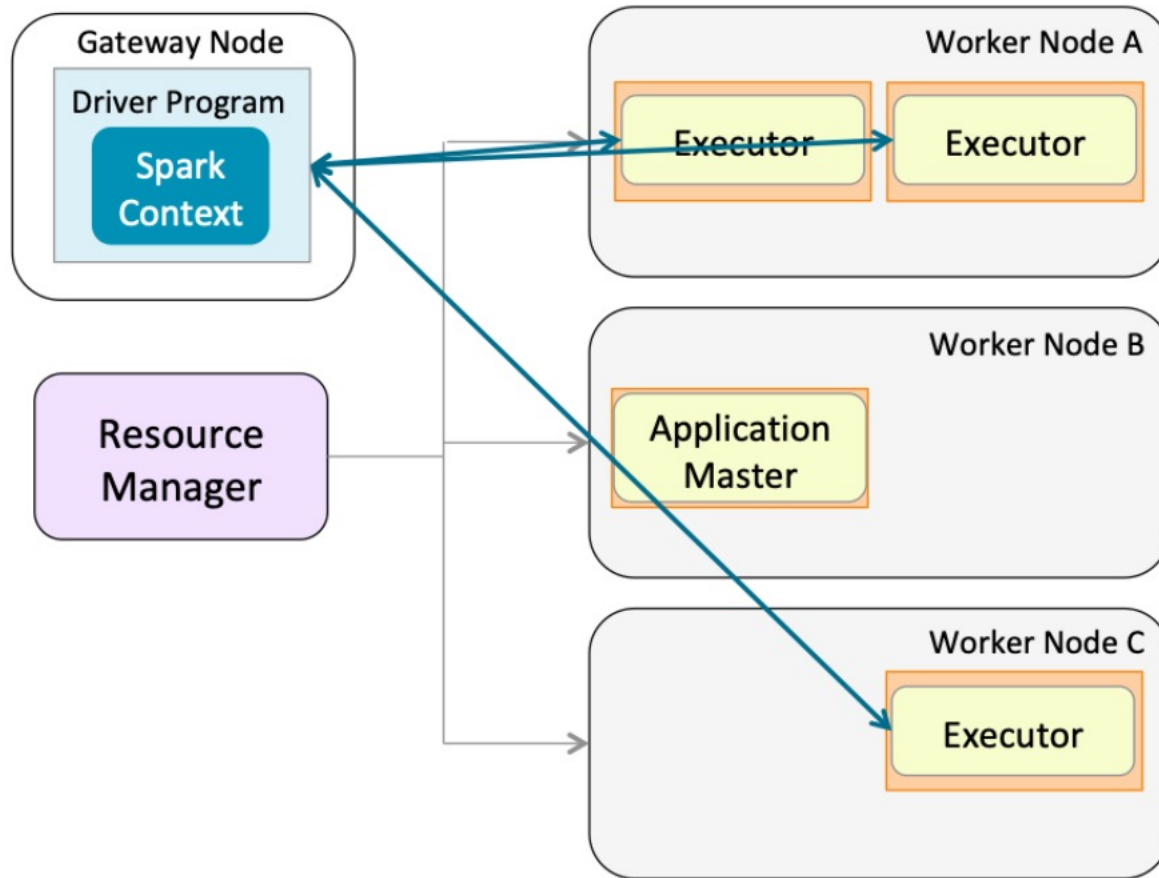
Spark Distributed Processing

Review of Spark on YARN (2)



Spark Distributed Processing

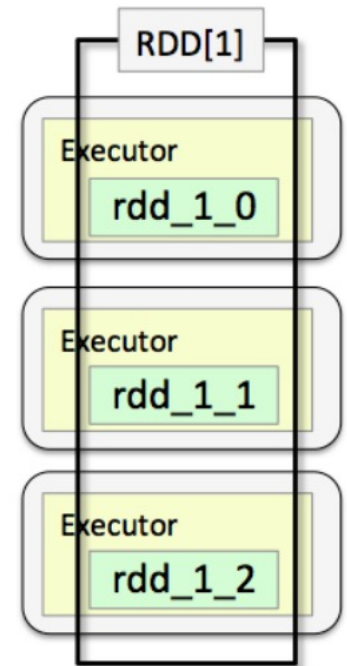
Review of Spark on YARN (3)



Spark Distributed Processing

Data Partitioning (1)

- **Data in Datasets and DataFrames is managed by underlying RDDs**
- **Data in an RDD is *partitioned* across executors**
 - This is what makes RDDs *distributed*
 - Spark assigns tasks to process a partition to the executor managing that partition
- **Data Partitioning is done automatically by Spark**
 - In some cases, you can control how many partitions are created
 - More partitions = more parallelism



Spark Distributed Processing

Data Partitioning (2)

- **Spark determines how to partition data in an RDD, Dataset, or DataFrame when**
 - The data source is read
 - An operation is performed on a DataFrame, Dataset, or RDD
 - Spark optimizes a query
 - You call repartition or coalesce

Spark Distributed Processing

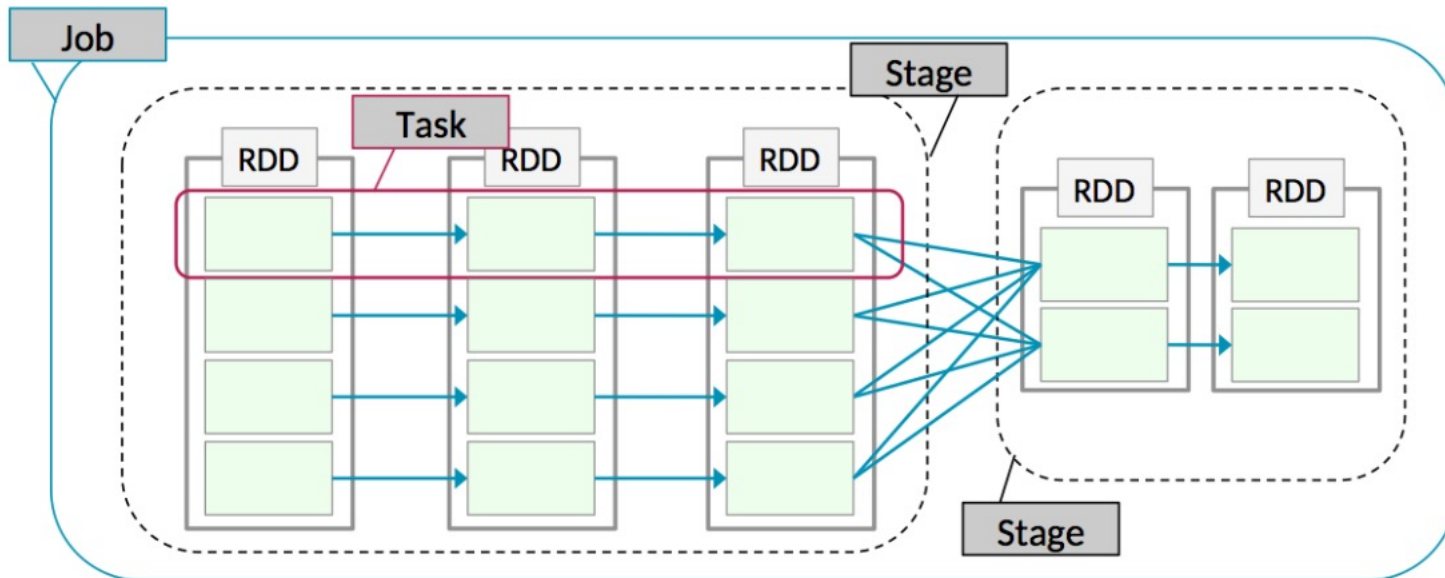
Stages and Tasks

- A *task* is a series of operations that work on the same partition and are pipelined together
 - *Stages* group together tasks that can run in parallel on different partitions of the same RDD
 - *Jobs* consist of all the stages that make up a query
 - Catalyst optimizes partitions and stages when using DataFrames and Datasets
- Core Spark provides limited optimizations when you work directly with RDDs
 - You need to code most RDD optimizations manually
 - To improve performance, be aware of how tasks and stages are executed when working with RDDs

Spark Distributed Processing

Summary of Spark Terminology

- **Job**—a set of tasks executed as a result of an *action*
- **Stage**—a set of tasks in a job that can be executed in parallel
- **Task**—an individual unit of work sent to one executor
- **Application**—the set of jobs managed by a single driver



Distributed Data Persistence

Persistence

- **You can *persist* a DataFrame, Dataset, or RDD**
 - Also called *caching*
 - Data is temporarily saved to memory and/or disk
- **Persistence can improve performance and fault-tolerance**
- **Use persistence when**
 - Query results will be used repeatedly
 - Executing the query again in case of failure would be very expensive
- **Persisted data cannot be shared between applications**

Distributed Data Persistence

Storage Levels: Partition Replication

- **Replication—store partitions on two nodes**
 - DISK_ONLY_2
 - MEMORY_AND_DISK_2
 - MEMORY_ONLY_2
 - MEMORY_AND_DISK_SER_2 (Scala and Java only)
 - MEMORY_ONLY_SER_2 (Scala and Java only)
 - You can also define custom storage levels for additional replication

Distributed Data Persistence

Default Storage Levels

- **The storageLevel parameter for the DataFrame, Dataset, or RDD persist operation is optional**
 - The default for DataFrames and Datasets is MEMORY_AND_DISK
 - The default for RDDs is MEMORY_ONLY
- **persist with no storage level specified is a synonym for cache**

myDF.persist()

Distributed Data Persistence

When and Where to Persist

- **When should you persist a DataFrame, Dataset, or RDD?**

- When the data is likely to be reused
- Such as in iterative algorithms and machine learning
- When it would be very expensive to recreate the data if a job or node fails

- **How to choose a storage level**

- **Memory**—use when possible for best performance
- Save space by serializing the data if necessary
- **Disk**—use when re-executing the query is more expensive than disk read
- Such as expensive functions or filtering large datasets
- **Replication**—use when re-execution is more expensive than bandwidth

Spark Structured Streaming

Real-Time Data Processing with Spark Structured Streaming

Content:

•What is Spark Structured Streaming?

A scalable and fault-tolerant stream processing engine built on the Spark SQL engine.

•Key Features:

- Processes streaming data using the same APIs as batch
- Supports **event time processing, watermarking, windowing**
- Integrated with Kafka, files, sockets, and more
- Output modes: **append, update, complete**

•Why use it?

- Unified batch + stream processing
- Declarative and optimized execution
- Exactly-once semantics (when supported)



"Treat streaming as a continuous table, not a microservice."

Streaming Architecture

Architecture Overview:

- Data sources: Kafka, files, socket, etc.
- Ingestion → Transformations → Sink
- Internally runs as **micro-batches** or **continuous mode**

